

Module 2: Design and Management Techniques

Session 4: Introduction to Jira & Test Management

Chapter 1: Jira and Agile Methodology (Corporate Language)	2
1. What is Jira?	2
2. Why do we use Jira? (Agile context)	2
Chapter 2: Jira Hierarchy (Epic, Story, Task, Bug)	3
1. The Epic (The Great Project)	3
2. User Story (Business Requirement)	3
3. The Task (Technical Task)	4
4. The Bug (The Defect)	4
5. The sub-task	4
💡 QualiAdept Analogy: "Building a Residential Neighborhood"	4
🧠 Thinking Exercise: "Frame the Ticket"	5
Chapter 3: Practical Guide - Build your own Jira lab.....	5
Step 1: Register for a free account.....	6
Step 2: Setup Assistant (Creating your first Board/Space).....	7
Step 3: Creating your first Ticket (User Story)	8
Step 4: QA "Magic" - Installing the Testing Plugin (Zephyr Scale).....	9
Chapter 4: Writing and Running the First Test Case (Zephyr Scale).....	11
Step 1: Creating the Test Case in Zephyr	11
Step 2: Writing the Steps (Test Script)	12
Step 3: The "Holy Grail" of QA - Traceability	13
Step 4: Test Execution (Test Cycle)	14
Step 5: Failure Scenario (Reporting a BUG)	16
💡 QualiAdept Analogy: "Traffic Policeman"	17

 What's next?.....	17
Chapter 5: Recap, Conclusions and Bonus Module (JQL & Dashboards).....	17
1. Comprehensive Review: What did we learn in Session 4?	18
 Bonus: "Secrets of a Senior QA in Jira"	18
A. JQL (Jira Query Language) - Searching Like a Hacker.....	18
B. Dashboards and Visual Reporting.....	19
C. Artificial Intelligence in Jira (Atlassian Intelligence / RoVo).....	19
 Final Practical Activity: "Configuring the Command Center"	20

Welcome to Module 2!

So far I have learned *how* to think like a QA and how *to* write a plan or a test case. But in a company with hundreds of employees, you can't keep these plans in a Word file on your desktop. You need a centralized system, visible to everyone.

This is where it comes into play **Jira**, the most popular software project management tool in the world.

Chapter 1: Jira and Agile Methodology (Corporate Language)

1. What is Jira?

Developed by Atlassian, Jira was originally created as a simple bug tracker. Today, it has evolved into a complete ecosystem where teams plan their work, assign tasks, track progress, and release software.

2. Why do we use Jira? (Agile context)

In the past (in the Waterfall model), software was built linearly: 6 months of analysis, 1 year of programming, 3 months of testing. Today, companies use the methodology **Agile** (and the framework **Scrum**).

- **The Sprint:** In Agile, work is divided into small "chunks" called Sprints (usually 2 weeks). At

the end of 2 weeks, the team must deliver a complete and tested functionality.

- **Jira role:** Jira is the board where we post all our ideas, requirements and bugs. It helps us instantly answer questions like: *"What do we have to do this sprint?"*, *"What is not working on now?"*, *"How many bugs do we have to fix by Friday?"*.

Chapter 2: Jira Hierarchy (Epic, Story, Task, Bug)

To avoid turning the project into chaos, Jira uses strictly hierarchical "Issues". Let's learn the daily vocabulary of any IT team:

1. The Epic (The Great Project)

- **Definition:** A massive piece of work that cannot be completed in a single Sprint (2 weeks). It is a collection of smaller requirements that have a common goal.
- **Example (OrangeHRM):** EPIC-01: Recruitment Module.
- **Who creates it?** The Product Owner or Project Manager.

2. User Story (Business Requirement)

- **Definition:** A software requirement written from the end user's perspective. Responds to the format:

"As [User type / Role],

I want to [Action / Functionality],

So that [Benefit / Added value]."

- This is the basic unit you will encounter most often as a QA. A Story must be able to be completed in a single Sprint.
- **Example (OrangeHRM):** STORY-05: As a Manager, I want to be able to upload a candidate's resume in PDF format.

Practical examples:

- **As a frequent buyer, I want to be able to save** products to a favorites list, **so that** I can buy them faster next time.
- **As a system administrator, I want to** receive an email alert when the server reaches 90%

capacity, **so that I** can prevent the application from crashing.

The complete structure of a User Story

In project management tools (such as **Jira** or Azure DevOps), an effective User Story is always accompanied by **Acceptance Criteria**, often written in BDD format (**Given-When-Then**):

- **Title:** [Role] - [Short functionality]
- **Description:** The standard format mentioned above (**As a... I want... In order to...**)
- **Acceptance criteria:**
 - *Given* (Initial context)
 - *When* (User action)
 - *Then* (Expected result)

3. The Task (Technical Task)

- **Definition:** A work task that needs to be done, but that does not bring direct or visible value to the end user.
- **Example:** TASK-12: Updating the database from version 1.4 to 1.5 or TASK-13: Creating documentation for the API.

4. The Bug (The Defect)

- **Definition:** Report created by the Tester (QA) when the software does not behave according to the User Story (the actual result does not match the expected result).
- **Example:** BUG-22: Error 500 when pressing the upload CV (PDF) button.

5. The sub-task

- **Definition:** It is the "child" of a Story, Task, or Bug. It represents the small steps needed to close the parent ticket.
- **Example:** For STORY-05, we can have:
 - Sub-task 1: (For Dev) Writing the code for the Upload button.
 - Sub-task 2: (For QA) Create test cases for CV upload.

💡 **QualiAdept Analogy: “Building a Residential Neighborhood”**

💡 Imagine Jira as the control panel of a construction company:

- **The Epic** is like a *Construction of a Block A* → It will take a year.
- **The User Story** is like *Apartment 12 on the 3rd floor*. We build it so that the user can move in. It has a living room, bathroom and kitchen.
- **The task** is like *digging the foundation or bringing the crane*. The tenant does not see these things, but without them the apartment cannot exist.
- **The sub-task** is like *installing tiles in the bathroom of apartment 12*. (A small activity to finish the Story).
- **The bug** is like the moment when the Quality Inspector (QA) opens the tap in Apartment 12 and finds that *the pipe is leaking under the sink...* It must be repaired before the tenant receives the keys!

Thinking Exercise: "Frame the Ticket"

Read the following tickets and decide if they are Epic, Story, Task or Bug:

- ❖ Complete Payment Processing Module (card, transfer, crypto). (_____)
- ❖ The "Log in" button is visible, but when I press it, nothing happens. (_____)
- ❖ As an unregistered user, I want to be able to view the product catalog to decide what to buy. (_____)
- ❖ Setting up the new test server for the QA team. (_____)

Chapter 3: Practical Guide - Build your own Jira lab

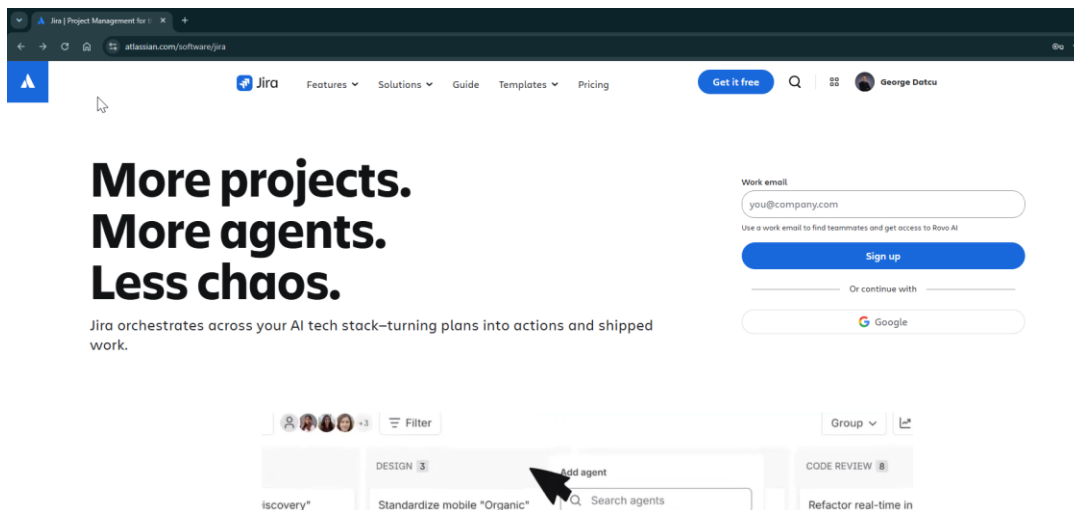
🎯 Now that we know what tickets and Agile methodology are, it's time to get our hands dirty. In corporate settings, you'll get a ready-made account. But for practice at home, we'll teach you how to create your own Jira instance, completely free, using Atlassian's new setup flow.

Follow these steps exactly as written. You don't need a credit card, just an email address.

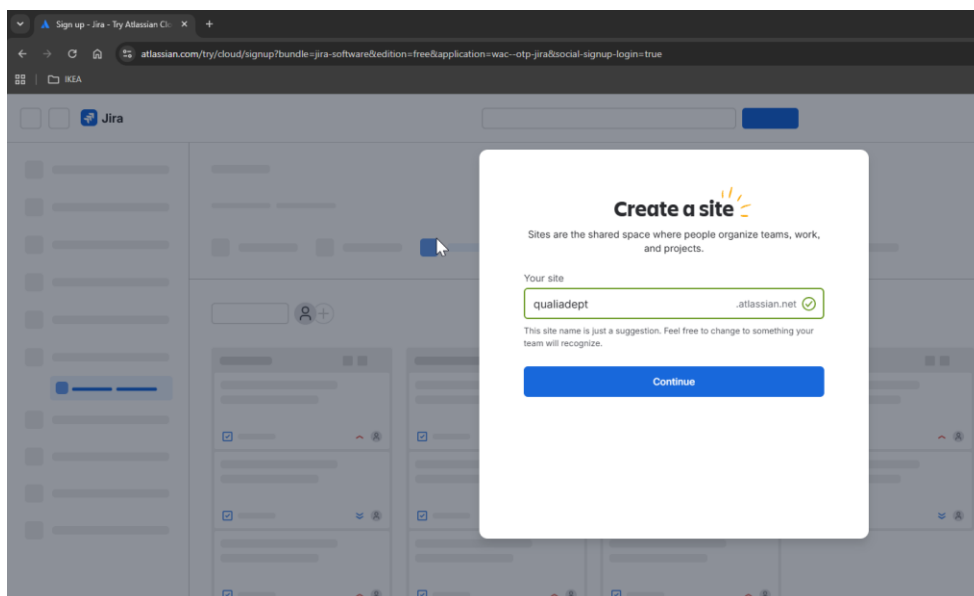
Step 1: Register for a free account

Jira offers a free plan for small teams (up to 10 users), perfect for your portfolio.

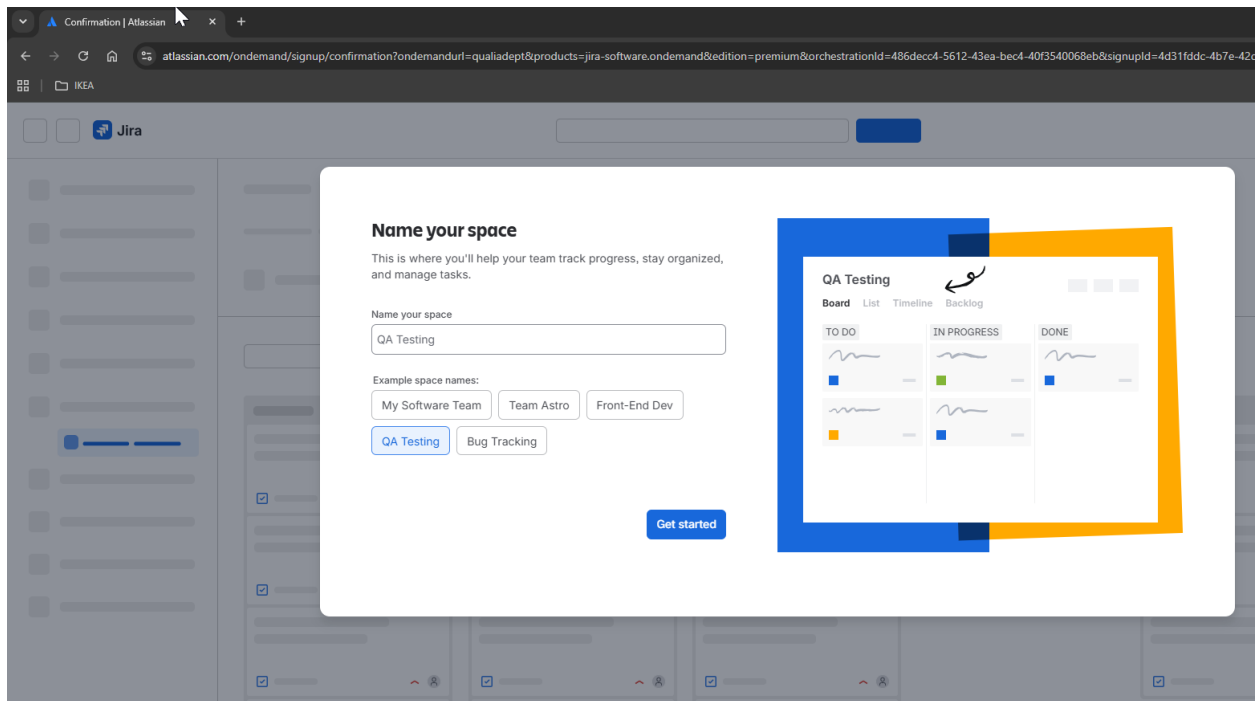
- **Access the website:** Enter on <https://www.atlassian.com/software/jira> and press the blue button „Get it free” (top of the menu).



- **Authentication:** In the "Get started with Jira" window, enter your email address and press **Sign up** (or choose "Continue with Google" for speed).



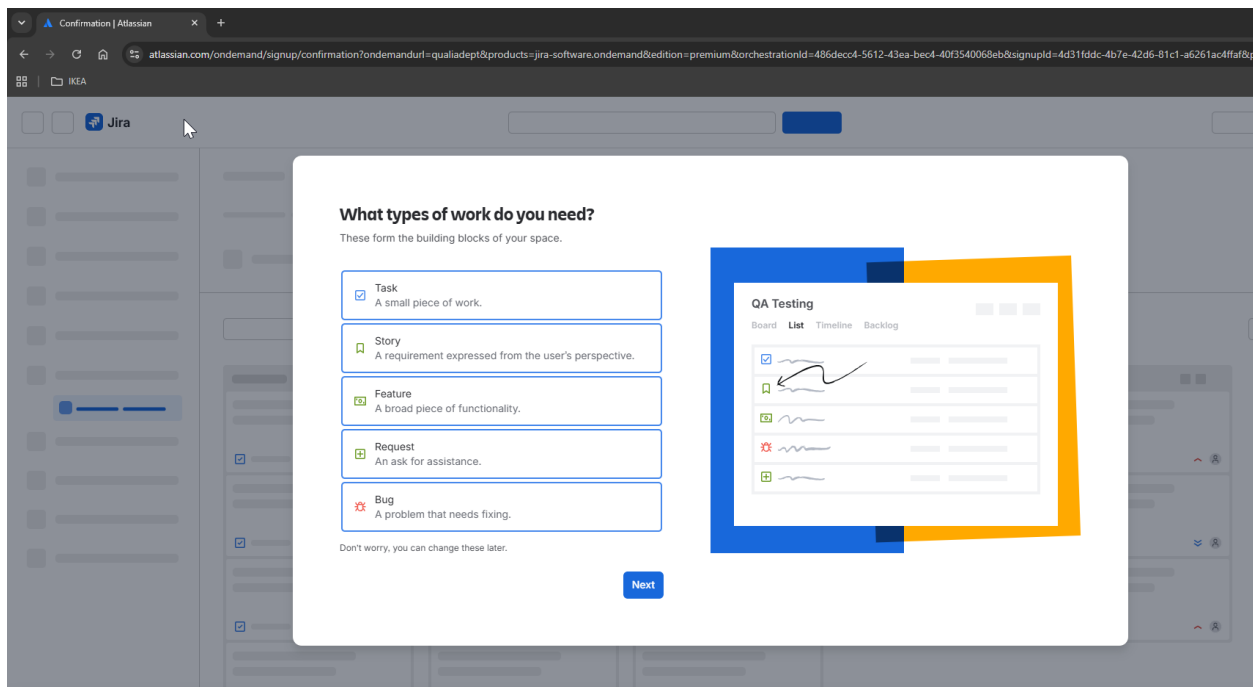
- **Name your site (Create a site):** Jira will ask you to choose a name for your web space (e.g. your-name-portfolio.atlassian.net). Choose a professional name, make sure the green checkmark appears, and click **Continue**.



Step 2: Setup Assistant (Creating your first Board/Space)

Atlassian has simplified project creation with an assistant that asks you how you want to work. Answer this to get the perfect Agile/Scrum board for QA:

- **What kind of work do you do?** (What type of work do you do?)
 - Choose **Software development**.
- **How does your team plan to use Jira?** (How does the team plan to use Jira?)
 - Check the option **Work in scrum** (and optional *Track bugs*). Press **Continue**.
- **Name your space** (Name your space/project):
 - This is where you actually create your test panel. Write in Name your space: **OrangeHRM QA** (or QA Testing). Press **Get started**.
- **What types of work do you need?** (What types of tickets do you need?)
 - Make sure at least the following are checked: **Task**, **Story** and **Bug**. These are the "bricks" you need to report defects and requirements. Click **Next**.



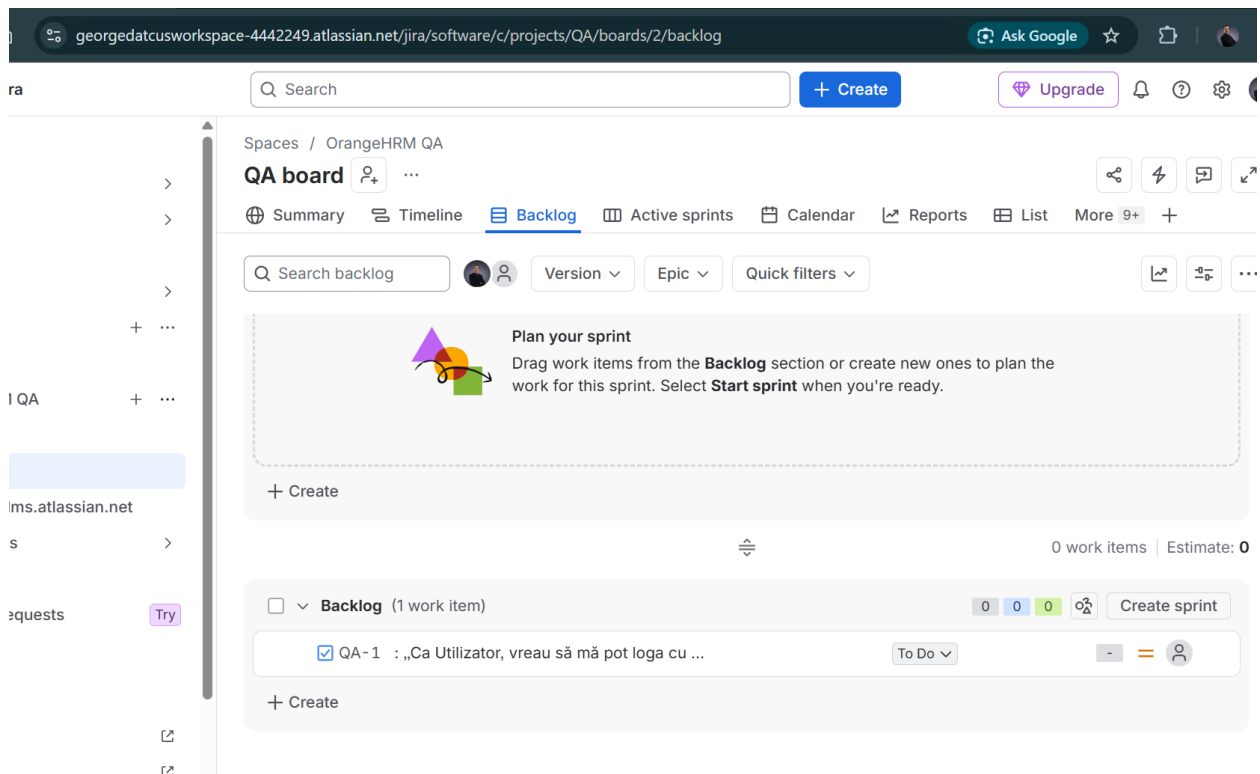
Done! The system will throw you directly into the menu **Backlog** of your new workspace.

Step 3: Creating your first Ticket (User Story)

If you look at your screen, you have a main menu on the left and a large work area in the center (the Backlog). Let's add a business requirement.

- Look at the top bar of the screen. Next to the Search bar, click the blue button **Create**.
- A new window will open (Issue Create).
- **Project/Space:** Make sure your space ("OrangeHRM QA" or "QA Testing") is selected.
- **Issue type:** Choose **Story** (Requirement).
- **Summary (Title):** Write: *"As a User, I want to be able to log in with valid credentials to access the dashboard"*.
- **Description:** Write the acceptance criteria. (Ex: 1. The system must accept the username 'Admin'. 2. The password must be masked.)
- Press the button **Create** from the bottom right corner.

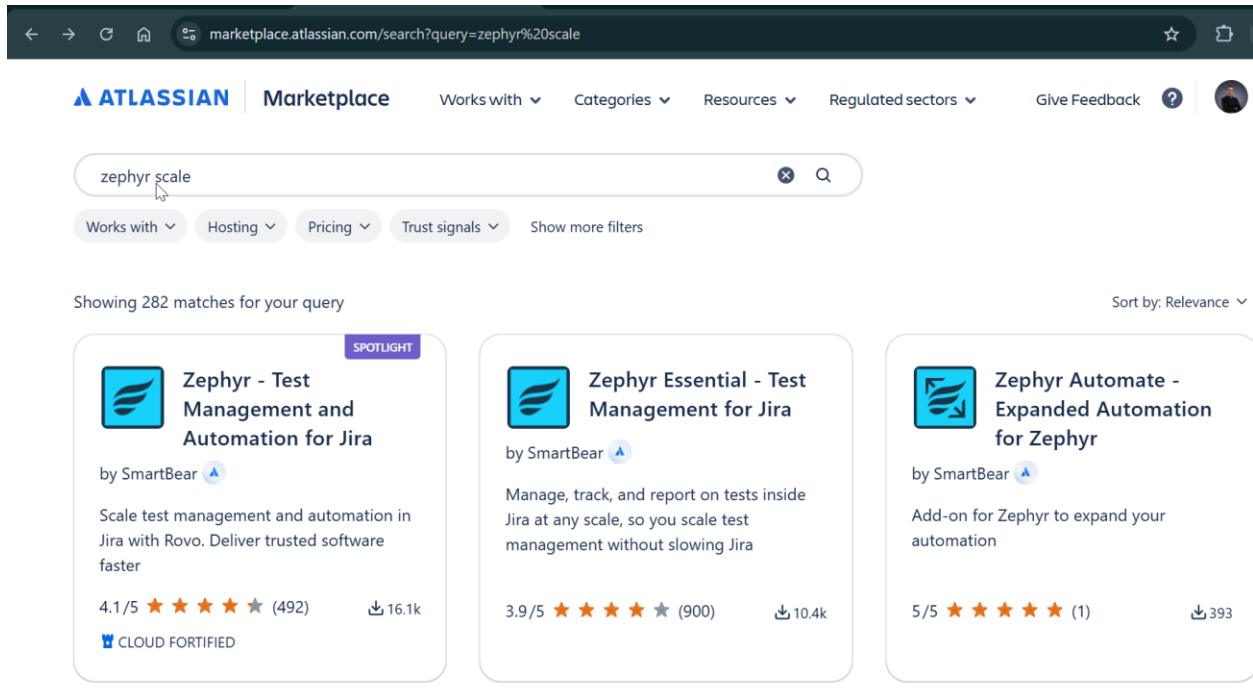
If you look in the central area of the screen, at the section **Backlog**, you will see your first ticket created and ready for the sprint!



Step 4: QA “Magic” - Installing the Testing Plugin (Zephyr Scale)

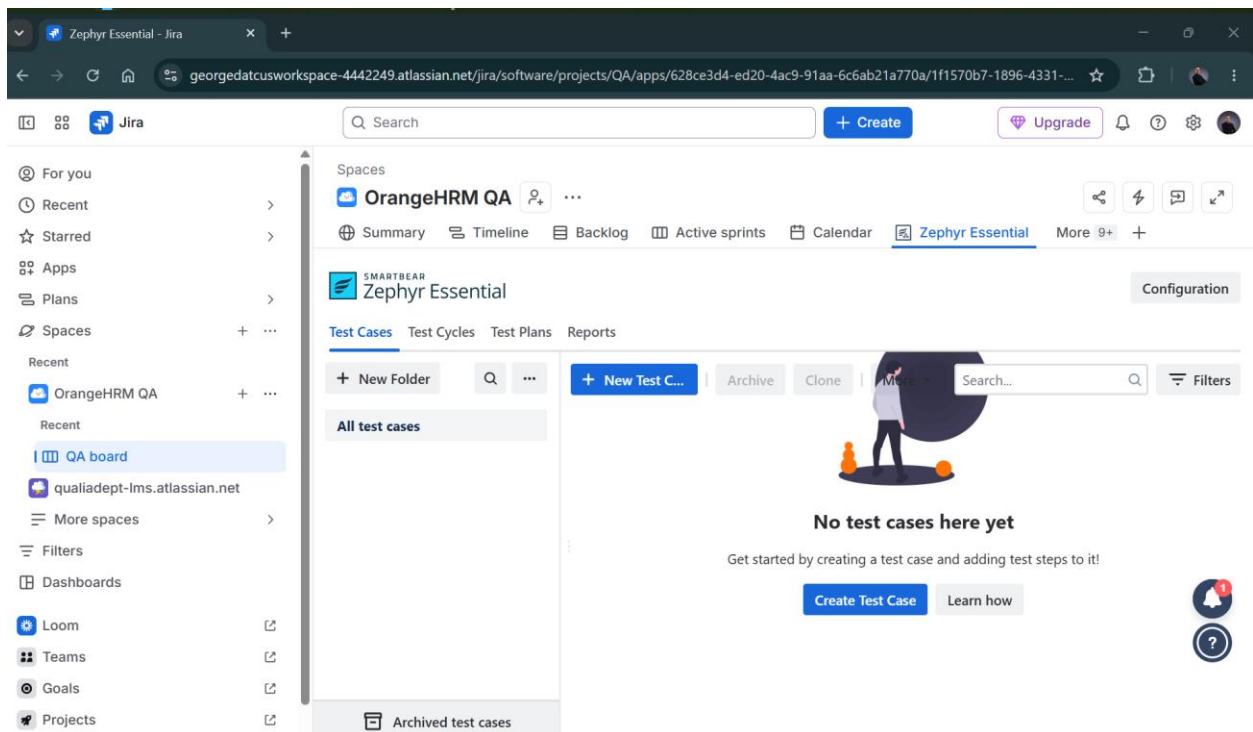
Standard Jira is perfect for moving tickets, but it doesn't have the *“Write Test Cases”* button. For this, we need to install an extension dedicated to testers. We will use **Zephyr Scale**, the most modern free tool.

- Look in the left **vertical menu** (the one with the dark background).
- Under the sections *Recent*, *Starred* etc., click on the section **Apps** (Applications).
- From the drop-down menu, choose **Explore more apps**.
- In the store search bar, type **Zephyr Scale**.
- Click on the app *Zephyr Scale - Test Management for Jira*.



- Press the button **Try it free** (or Get app). It's 100% free for your account.
- Wait for the installation and return to your space (OrangeHRM QA).

Success test: The option should now appear in your workspace menu. **Zephyr Scale** - If it's there, your QA lab is fully configured and ready for action!

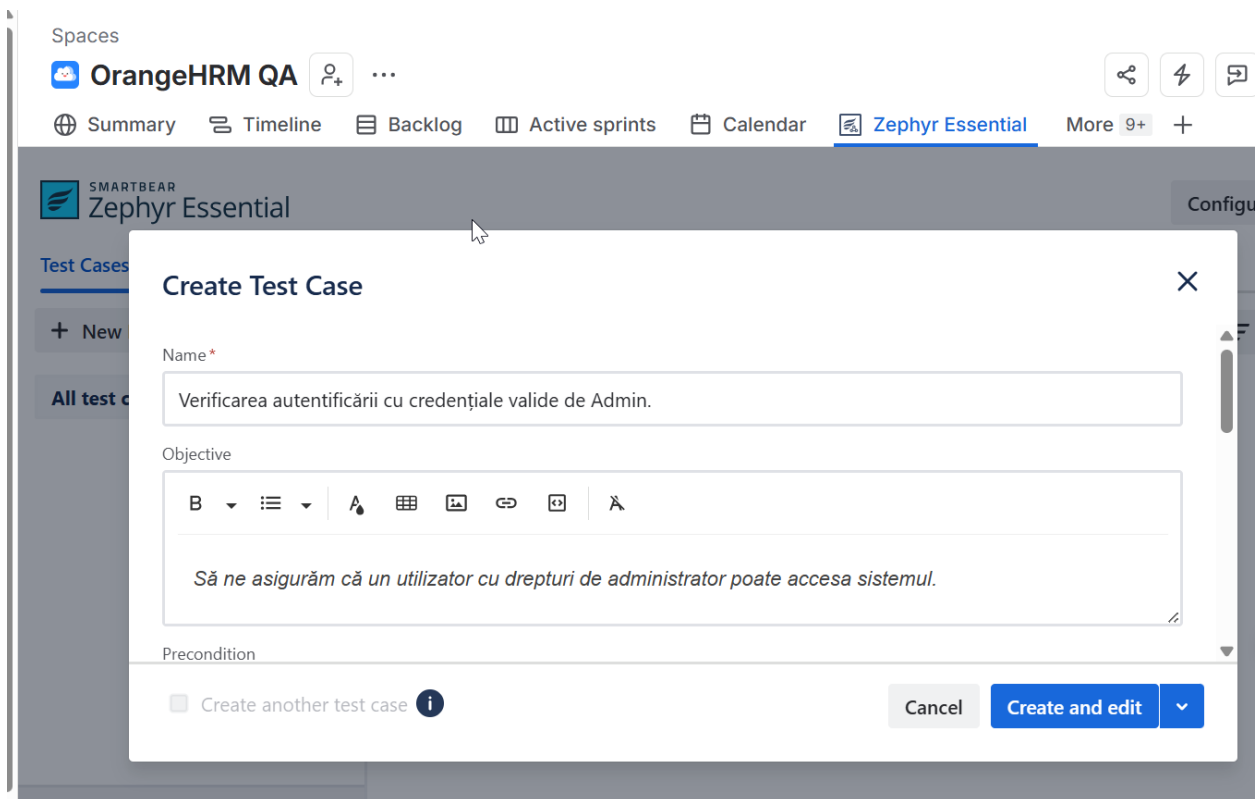


Chapter 4: Writing and Running the First Test Case (Zephyr Scale)

 If Jira is the project dashboard, **Zephyr Scale** is your personal tester's diary. In this chapter, we will take the requirement created in the previous step (the User Story with Login) and "cover" it with a professional test case.

Step 1: Creating the Test Case in Zephyr

- In the left menu of your "OrangeHRM QA" project, click **Zephyr Scale**.
- From the plugin's top menu, select the section **Tests**.
- Press the blue button **New Test Case** from the top right.
- A detailed form will open. Let's fill it out:
 - **Name:** *Verifying authentication with valid Admin credentials.*



- **Objective (optional):** *Let's make sure that a user with administrator rights can access the system.*

Create Test Case

B
:≡
A
Table
Image
Link
Code
A

Să ne asigurăm că un utilizator cu drepturi de administrator poate accesa sistemul.

B
:≡
A
Table
Image
Link
Code
A

Precondition (Precondiție): *Utilizatorul trebuie să aibă un cont activ de Admin și să fie pe pagina de Login (URL: hrm.orange.com/login).*

- **Precondition (Precondiție):** *The user must have an active Admin account and be on the Login page (URL: hrm.orange.com/login).*

Step 2: Writing the Steps (Test Script)

This is where the execution work comes in. Click on the tab **Test Script** (located immediately below

the test name).

You will see a table with actions (Test Step) and expected results (Expected Result). Let's add the steps:

- **Step 1:**
 - *Test Step:* Enter the text Admin in the Username field.
 - *Expected Result:* The text is entered successfully.
 - Press *Add Step* (or Enter).
- **Step 2:**
 - *Test Step:* Enter the correct password in the Password field.
 - *Expected Result:* The password is masked (asterisks/dots appear).
- **Step 3:**
 - *Test Step:* Click on the "Login" button.
 - *Expected Result:* The system accepts the data and the user is redirected to the main page (Dashboard).

Spaces

OrangeHRM QA

Summary Timeline Backlog Active sprints Calendar Zephyr Essential More 9+ +

OrangeHRM QA / Test Cases / Verificarea autentificării cu credențiale... (QA-T1)

Verificarea autentificării cu credențiale valide de Admin.

Details Test Script Traceability Execution History

TEST DATA

None

EXPECTED RESULT

Sistemul acceptă datele și utilizatorul este redirecționat către pagina principală (Dashboard).

+ Add step

Hint: when editing the last step, press **Tab** to add a new one.

You have unsaved changes

Discard changes

Save test script

Step 3: The "Holy Grail" of QA - Traceability

This is the most important step a QA Manual takes in Jira. We need to demonstrate to the manager why we wrote this test. We need to *tie* of the initial requirement.

- Click on the tab **Traceability** (Next to Test Script).
- To the section/Issues, press **Link Issues** (or the search box).
- Find the User Story you created in Chapter 3 ("As a User, I want to be able to log in...").
- Select it and press **Link**.
- Finally, press the button **Save** in the top right to save the entire test case.

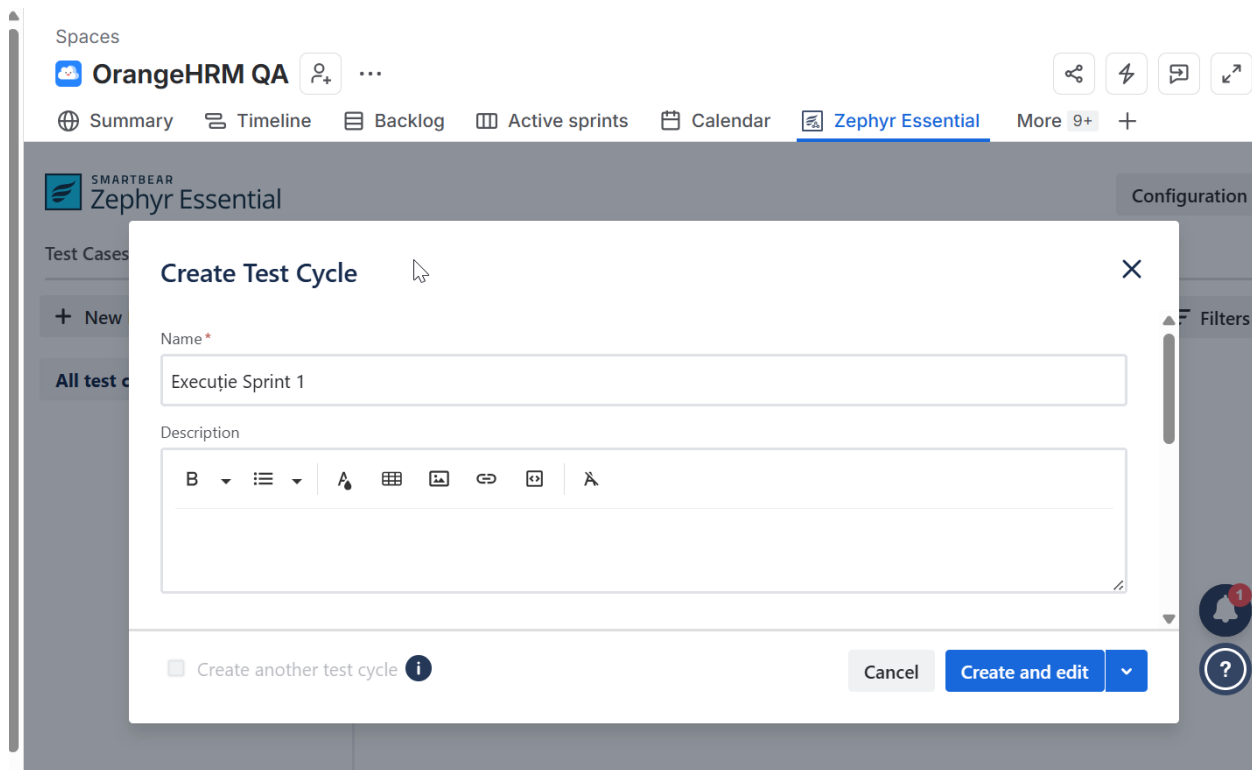
The screenshot shows the Jira Zephyr Essential interface. At the top, there's a 'Spaces' section with 'OrangeHRM QA' selected. Below this is a navigation bar with tabs: Summary, Timeline, Backlog, Active sprints, Calendar, Zephyr Essential (selected), and More 9+. The main content area shows the test case 'Verificarea autentificării cu credențiale valide de Admin.' under the 'Test Cases' section. The 'Traceability' tab is active, showing a section for 'Issues(1)' with a 'Create new issue' button and an 'Add existing issue' button. Below this, a 'Coverage' section shows a linked issue 'QA-1 : „Ca Utilizator, vreau să mă pot loga cu credențiale valide pentru a accesa dashboard-ul”' with a 'TO DO' status. At the bottom, there's a 'Web Link (0)' section with an 'Add Web Link' button.

Congratulations! Now, if someone opens the User Story, they will see your Test Case attached. They know that that requirement is "covered" (Test Coverage).

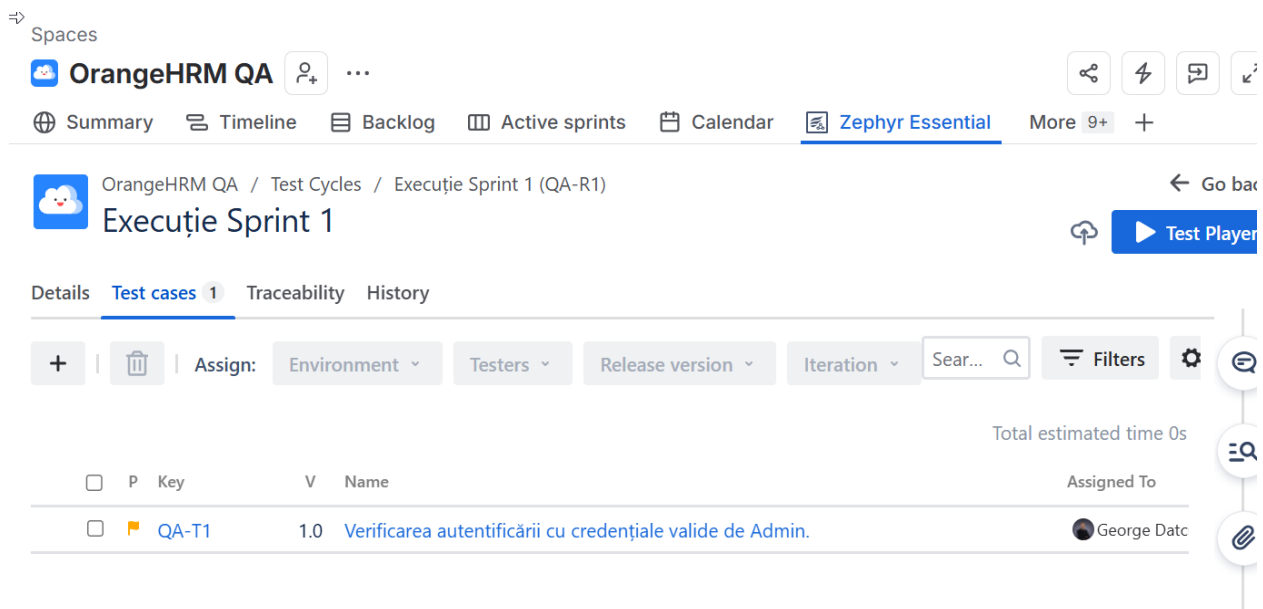
Step 4: Test Execution (Test Cycle)

Let's say the developer tells you: "I've finished programming the login, you can test it! "You don't just open the site and test chaotically, you start a "Run".

- In Zephyr, go to the tab **Test Cycles** (Test Cycles). A cycle is basically a "folder" where we collect the tests we want to run today.
- Press **New Test Cycle**. Name it: *Sprint Execution 1*. Press Save.



- Enter this Test Cycle, go to the tab **Test Cases** and press **Add Test Cases**. Select your login test and add it.



- Now, next to the test, you will see the status "Not Executed". Click the button **Play (Start Execution)**.
- The step execution window will open!

Here, as QA, you start doing the steps on the real application (or test environment). For each step in your script, you have status buttons:

- ❖ ☒ **Pass:** Worked perfectly.
- ❖ ☒ **Fail:** The application did something else (error).
- ❖ ☒ **Blocked:** You cannot test the step (e.g. the server is down or you have no internet).

Spaces

OrangeHRM QA

Summary Timeline Backlog Active sprints Calendar Zephyr Essential More 9+ +

OrangeHRM QA / Test Cycles / Execuție Sprint 1 (QA-R1) / Test Player

Execuție Sprint 1

No estimated time • No execution time • Planned Start Date: 02/Aug/26 • Planned End Date: 02/Aug/26

Test Cases 1

Search...

Group by: Show assigned to me 1

1 test case

QA-T1 (1.0) Verificarea autentificării cu credențiale valide de Admin

000:00:05 Set all below to:

Start a new test execution

QA-T1 (1.0) Verificarea autentificării cu credențiale valide de Admin

Pass 000:00:05

No estimated time • Actual time: 5s • 02/Aug/26 12:38 PM • History

Execution Details

Environment

None

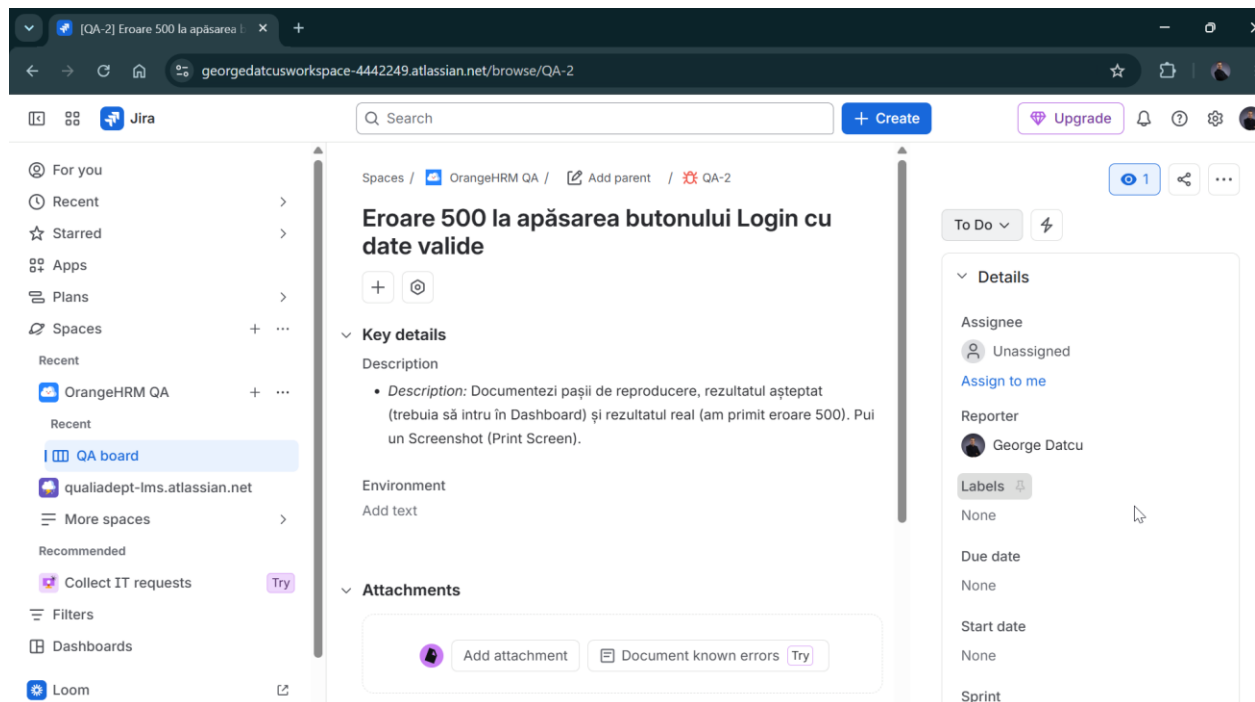
Iteration

None

Step 5: Failure Scenario (Reporting a BUG)

What do we do if at Step 3 (click Login), the application gives us the error "Server Error 500"?

- In Step 3 in Zephyr, you change the status from *Not Executed* to **Fail** ☒.
- Zephyr is smart: it will immediately show you a button **Create Issue..**
- Click it and write the details of the Bug:
 - *Summary:* Error 500 when pressing the Login button with valid data.
 - *Description:* Document the replication steps, the expected result (I had to log into the Dashboard), and the actual result (I got a 500 error). Take a Screenshot (Print Screen).
- Click Create.



Jira Magic: This Bug is automatically linked to your Test Case, which is linked to the User Story. This way, the Product Owner can see directly from the Story: *We have an open bug, so we can't release the functionality to customers!*

💡 QualiAdept Analogy: “Traffic Policeman”

- **The User Story** is the law that says: *"Cars must stop at red lights"*.
- **The Test Case (Script)** is the radar you install: *"I measure whether car X stops at the traffic light"*.
- **The Traceability** is the link in the file: *"I installed this radar TO check the red light law"*.
- **The IMPLEMENTATION** it's the moment when you actually look at the traffic light.
- **The Bug** is the fine you write if the car (the application) runs a red light!

🚀 What's next?

With this knowledge, you can already be functional in an IT company from day one! You learned to read requirements, write clear steps, execute them and bug-proof the problems found. This is the life cycle of a manual tester!

Chapter 5: Recap, Conclusions and Bonus Module

(JQL & Dashboards)

🎉 We've reached the end of one of the most practical and important sessions in this course. You learned not only the theory of Agile, but also how to set up your own workspace in Jira, the ultimate tool used by IT corporations around the world.

1. Comprehensive Review: What did we learn in Session 4?

- **Agile language:** We learned that we no longer work with documents of hundreds of pages, but with clearly hierarchized tickets (Issues): **Epic**(big goal) → **User Story** (customer requirement) → **Task / Sub-task** (technical work) → **Bug** (the defect).
- **Environment Configuration:** I went through the modern Jira interface (2026), setting up a project/space of type **Scrum**, optimized for testing (QA Testing).
- **Test Management:** I installed and used the extension **Zephyr Scale** We now know that a Test Case written in Excel is a relic of the past.
- **Traceability:** I learned the golden rule of modern testing: *No test should exist "in the air"*. We connected (mapped) our Test Case directly to the User Story in Jira, to prove requirements coverage.
- **Execution (Test Cycle):** We grouped the tests into an execution cycle, ran them, and marked the steps with "Pass" or "Fail".

🎁 Bonus: "Secrets of a Senior QA in Jira"

A novice tester clicks through menus to find what they're looking for. A QualiAdept tester uses the platform's advanced tools to be 10x faster. In this bonus module, we explore essential functionality in today's IT landscape.

A. JQL (Jira Query Language) - Searching Like a Hacker

JQL is the language through which you can query the Jira database to find exactly the tickets you need. It is used in the top search bar (section *Filters* -> *Advanced issue search*).

- **Example 1** (What do I have to test today?):

You want to see all User Stories that are in "IN QA" status and are assigned to you.

```
status = "IN QA" AND assignee = currentUser()
```

- **Example 2 (Bug Hunt):**
Do you want to see all the Bugs you created this week that have not yet been fixed by the developers?

```
issuetype = Bug AND reporter = currentUser() AND status != Done AND created  
>= startOfWeek()
```

💡 **QualiAdept tip:** Once you have written a good JQL, you can save it as a **Filter(Filter)**. So next time you only need one click to run the same search!

B. Dashboards and Visual Reporting

As a QA, you will often need to present the quality status to management. Managers don't have time to read 100 Test Cases; they want graphs.

- **Creating a Dashboard:** From the Jira main menu → *Dashboards* → *Create dashboard*.
- **Adding Gadgets (Widgets):**
 - **Pie Chart:** You can create a graph that shows you how many of the 50 reported bugs are Critical, Major, or Minor.
 - **Zephyr Scale Metrics:** Because you have Zephyr installed, you have special gadgets (ex: *Test Execution Results*). With a single graph you show how many tests were passed (Pass), failed (Fail) or unexecuted (Unexecuted) in the current sprint.

C. Artificial Intelligence in Jira (Atlassian Intelligence / Rovo)

In the Atlassian ecosystem of 2026, AI is natively integrated (e.g.: *Rovo AI* or *Atlassian Intelligence* - A modern QA uses these tools to save time:

- **Generating summaries:** An Epic has dozens of technical comments? Press the button *Summarize* of the AI, and it will give you in 3 paragraphs the necessary context to know what you have to test.
- **Writing assistance:** When writing a Bug Report, you can ask your assistant to rephrase your steps to be clearer, more polite, and more professional.
- *Careful:* The AI doesn't test for you. Critical thinking and hunting for edge-case scenarios

remain your sole responsibility.

Final Practical Activity: "Configuring the Command Center"

To reinforce the concepts from this session, access your newly created Jira Space and perform the following tasks:

- **Ticket** **Hunter:**
Navigate to *Filters*  *Advanced issue search*, switch to JQL mode and write a query that returns you **only** Story tickets. Save the filter with the name "All Stories".
- **Quality** **Manager:**
Create a *Dashboard* named "QA Metrics". Add a gadget like **Pie Chart** and configure it to use the filter saved in the previous step, grouping the results by *Status*.
- **Explorer** **Zephyr:**
in
Go to the Zephyr Scale module and generate a **Traceability Matrix** (Traceability Matrix). Analyze the report: which tickets have no associated test cases? That's where your biggest risk lies!

 **Message from the QualiAdept team:** Knowing the Jira platform transforms you from a simple executor to a strategist. Now you not only know how to test, but you know how to prove your work and manage the entire technological flow!